

Arrays

- Fixed size.
- Store elements of the same type.

```
int[] numbers = {1, 2, 3, 4};
```

ArrayList

- Resizable array.
- Allows random access.
- Part of `java.util` package.

```
import java.util.ArrayList;

ArrayList<String> list = new ArrayList<>();
list.add("Apple");
list.add("Banana");
```

LinkedList

- Elements are doubly linked.
- Efficient for insertions/deletions.

```
import java.util.LinkedList;

LinkedList<Integer> list = new LinkedList<>();
list.add(1);
list.addFirst(0);
```

Stack

- LIFO (Last In First Out).
- Use `Deque` for stack operations.

```
import java.util.Stack;
```

```
Stack<Integer> stack = new Stack<>();  
stack.push(10);  
stack.pop();
```

```
import java.util.ArrayDeque;
```

```
Deque<Integer> stack = new ArrayDeque<>();  
stack.push(1);  
stack.pop();
```

Queue

- FIFO (First In First Out).
- Implemented using `LinkedList` or `ArrayDeque`.

```
import java.util.LinkedList;  
import java.util.Queue;
```

```
Queue<String> queue = new LinkedList<>();  
queue.add("A");  
queue.poll();
```

PriorityQueue

- Elements are ordered based on priority (min-heap by default).

```
import java.util.PriorityQueue;

PriorityQueue<Integer> pq = new PriorityQueue<>();
pq.add(5);
pq.add(1);
pq.poll(); // 1
```

HashMap

- Stores key-value pairs.
- $O(1)$ average-case lookup and insertion.

```
import java.util.HashMap;

HashMap<String, Integer> map = new HashMap<>();
map.put("apple", 2);
map.get("apple");
```

HashSet

- Stores unique elements.
- Backed by a `HashMap`.

```
import java.util.HashSet;

HashSet<Integer> set = new HashSet<>();
set.add(1);
set.contains(1);
```

TreeMap & TreeSet

- Sorted versions of `HashMap` and `HashSet`.
- Backed by Red-Black Tree ($O(\log n)$ operations).

```
import java.util.TreeMap;
```

```
TreeMap<String, Integer> treeMap = new TreeMap<>();  
treeMap.put("banana", 1);  
treeMap.put("apple", 2); // sorted by keys
```

Deque (Double Ended Queue)

- Can add/remove from both ends.

```
import java.util.ArrayDeque;
```

```
Deque<Integer> deque = new ArrayDeque<>();  
deque.addFirst(1);  
deque.addLast(2);
```

Tree Structures

- **Binary Tree**
- **Binary Search Tree (BST)**
- **AVL Tree** (Self-balancing BST)
- **Red-Black Tree** (used in `TreeMap` and `TreeSet`)
- **Segment Tree**
- **Fenwick Tree (Binary Indexed Tree)**

These are not built-in but are often implemented for algorithmic problems.

Graphs

- Represented using:
 - **Adjacency List** (usually a `HashMap<Vertex, List<Edge>>`)
 - **Adjacency Matrix** (2D array)
 - Useful for BFS, DFS, Dijkstra, A*, etc.
-

Trie (Prefix Tree)

- Efficient for autocomplete, dictionary, and string matching.
 - Not in JCF; must be implemented manually.
-

Disjoint Set (Union-Find)

- Used for Kruskal's algorithm, cycle detection, etc.
-

Heap / Priority Queue Variants

- **Min Heap** and **Max Heap**
 - Java's `PriorityQueue` is a min-heap by default. Custom comparator can turn it into a max-heap.
-

Multimap / Multiset

- Not part of core JCF, but available in **Guava** or **Apache Commons**.
 - `Multimap<String, List<Integer>>` – one key, multiple values.
-

Concurrent Data Structures (Thread-safe)

From `java.util.concurrent`:

- `ConcurrentHashMap`
 - `CopyOnWriteArrayList`
 - `BlockingQueue` (e.g., `LinkedBlockingQueue`, `ArrayBlockingQueue`)
 - `ConcurrentLinkedQueue`, `ConcurrentSkipListMap`
-

Immutable Collections (Java 9+)

```
List<String> list = List.of("a", "b");
Set<String> set = Set.of("a", "b");
Map<String, String> map = Map.of("a", "1", "b", "2");
```

Deque Implementations

- `LinkedList`
 - `ArrayDeque`
 - Also supports stack and queue operations.
-

Specialized Libraries

- **Guava:** `Multimap`, `Multiset`, `Table`, `ImmutableList`, etc.
 - **Apache Commons Collections:** Bidirectional maps, fixed-size lists, etc.
 - **Eclipse Collections:** Richer data structure APIs.
-

Collection

- └─ `List` (`ArrayList`, `LinkedList`)
- └─ `Set` (`HashSet`, `TreeSet`, `LinkedHashSet`)
- └─ `Queue` (`PriorityQueue`, `LinkedList`, `ArrayDeque`)

Map

- └─ `HashMap`
- └─ `TreeMap`
- └─ `LinkedHashMap`